

IEEEXtreme 18.0 - Official Editorial

October 27th, 2024

IEEEXtreme 18.0 Technical Team:

- Racsó Galván - Technical Lead
- Andrei Grigorean - CS Academy
- Mihai Ciucu - CS Academy
- Douglas Gischlar - Problem setter
- Oded Margalit - Problem setter
- Nafis Sadique - Problem setter
- Jauhar Wibisono - Problem setter
- Philip Garabandic - Problem setter
- Miguel Mini - Problem setter
- Iván Bórquez - Problem setter
- Sebastian Sánchez - Problem setter
- Sachin Bharadwaj - Tester

Contents

Problem 1: Two fridges **3**

Problem 2: Star road **3**

Problem 3: Increasing table **3**

Problem 4: Bounded tuples **4**

Problem 5: Laser defense **4**

Problem 6: Another sliding window problem **5**

Problem 7: IEEE754 Emulator **6**

Problem 8: Triumvirates **6**

Problem 9: Stick **7**

Problem 10: Increasing-decreasing permutations **7**

Problem 11: Cheap Construction **7**

Problem 12: Disparate Data Sets **8**

Problem 13: Queries **8**

Problem 14: Icarus **9**

Problem 15: Doubled Sequence **9**

Problem 16: Power of three **12**

Problem 17: Halving **12**

Problem 18: King's Order **13**

Problem 19: Balls **14**

Problem 20: Corporation **15**

Problem 21: This is not an optimization problem **15**

Problem 22: Digits swap **17**

Problem 23: Brick stacks **17**

Problem 24: Stones **17**

Problem 25: Rectangles and arrays **18**

Problem 26: Invertible Pairs **18**

Problem 27: Sierpinski **19**

Problem Tutorial: "Two fridges"

Author(s): CS Academy

Keywords: **Implementation, brute force**

For this problem suffices to iterate through all the possible pairs of temperatures for both fridges and check if every substance's interval contains at least one of them.

This will have a time complexity of $O(M^2N)$, where $M = 100 - (-100) + 1 = 201$ and $N \leq 100$.

Problem Tutorial: "Star road"

Author(s): Iván Borquez

Keywords: **Data structures, dynamic programming, small to large trick**

To solve this problem, we can calculate the LIS (Longest Increasing Subsequence) and LDS (Longest Decreasing Subsequence) that *enter* and *exit* each node u. This allows us to obtain the optimal answer for node u using the formula:

$$\text{ans}(u) = \max\{\text{LDS_enter}(u) + \text{LIS_exit}(u), \text{LIS_enter}(u) + \text{LDS_exit}(u)\}$$

Since directly calculating these values might be challenging, we associate each possible path with the LCA (Lowest Common Ancestor) of its two nodes. This lets us use an arbitrary node as the root and focus on calculating the required information within the subtree of u for all nodes.

To achieve this:

1. First, gather information from each child subtree (using a segment tree and DFS timestamps) to calculate the LDS and LIS for u with respect to its subtree.
2. Finally, compute the cross-child results using the Small-to-Large trick on trees and a segment tree over the DFS timestamps.

The total complexity of this approach is $O(N \log^2 N)$.

Problem Tutorial: "Increasing table"

Author(s): CS Academy

Keywords: **Dynamic Programming**

For this problem we can define a function $dp(len_1, len_2)$ which means in how many ways we can fill the first len_1 cells of the first row and the first len_2 cells of the second row using only numbers from 1 to $len_1 + len_2$ and adhering to all the rules.

The dynamic programming algorithm must consider the following conditions over the states:

- len_1 must not be less than len_2, since that would break the column inequality condition.
- len_1 and len_2 must not exceed N.

So, we end up with the following recurrence:

$$dp(len_1, len_2) = \begin{cases} 0 & \max\{len_1, len_2\} > N \text{ or } len_1 < len_2 \\ 1 & len_1 = N \text{ and } len_2 = N \\ dp(len_1 + 1, len_2) & len_1 + len_2 + 1 \in A \\ dp(len_1, len_2 + 1) & len_1 + len_2 + 1 \in B \\ dp(len_1 + 1, len_2) + dp(len_1, len_2 + 1) & len_1 + len_2 + 1 \notin A, B \end{cases}$$

Notice that the additions are made modulo 998244353. The time complexity is $O(N^2)$.

Problem Tutorial: "Bounded tuples"

Author(s): CS Academy

Keywords: **Inclusion-exclusion principle, dynamic programming**

We refine the values of $high_i$ to obtain sharper inequalities. Using the inclusion-exclusion principle, the problem can be represented as 2^M instances, where each instance only has strict upper bounds on the sums. To manage these bounds, we process the bitwise addition of variables within each inequality.

To evaluate whether the sum satisfies sum $\leq R$, we represent R as a binary number and compute bit by bit from least significant to most significant, maintaining:

- A carry value in the range $[0, 2]$, which represents overflow from the current bit addition.
- A state indicating whether the current sum is $> R$ (greater) or $\leq R$ (less or equal).

We define a DP table dp[bit_pos][state][carry], where:

- bit_pos represents the number of processed bits.
- state is either $>$ or $\leq$ relative to R.
- carry tracks overflow from previous additions.

The transitions between states depend on the binary representation of R:

- Starting in the $\leq$ state, we stay there unless all previous bits are equal and we take a 1 while R takes a 0, transitioning to $>$.
- Starting in the $>$ state, we stay there regardless of the bits chosen.

For N variables, there are 2^N possible assignments of bits at each position. For each, we check 2^M inequalities. Each inequality requires 2 states $(>, \leq)$ and 3 carry values $(0, 1, 2)$, leading to 6^M possible combinations of states across M inequalities. Considering 64 bits for the binary representation, the total number of states is 64×6^M, and the transitions are computed in $O(2^N)$ for each DP state.

Thus, the overall complexity is:

$$O(64 \times 2^{N+M} \times 6^M)$$

Problem Tutorial: "Laser defense"

Author(s): CS Academy

Keywords: **Polar order sorting, two pointers**

For this problem we must notice that we can assume some ordering in the beams, so we will assume that the beams that are sent from B will be the first ones. That means that we currently have $M + 1$ regions inside our square.

Then, if we add a beam from A, we will notice that if it has an intersection with k beams from B such that their intersection point lies strictly inside the square, then the $k + 1$ regions involved will be doubled. This suggests that we will only need to compute the number of intersections from the beams of A and add them to the answer.

It's possible to compute the sum of the number of intersections in $O(N \log N)$ by sorting the lasers of A and B in clockwise order (as if they were vectors) and then using two pointers.

Problem Tutorial: "Another sliding window problem"

Author(s): Miguel Miní

Keywords: **Two pointers**

Before solving the problem, we must first analyze the functions R and E. For even n, we can see that the function R divides the sets into groups of two elements. Therefore, E attempts to find an optimal pairing such that the cost of placing two elements in the same set is given by the sum of the individual values.

Claim 6.1. *The optimal cost in a non-decreasing sequence $a_1, a_2, \ldots, a_m$ (with an even number of elements) is achieved by pairing the elements a_i and a_{m-i+1}.*

Proof. We can prove this by the exchange argument. Since the elements are sorted, suppose that some element among the first $\frac{m}{2}$ is paired with another element from the first half. Then, by the pigeonhole principle, some pair among the last $\frac{m}{2}$ elements will also be paired together. However, if we exchange the larger elements in each pair, our solution remains optimal. Hence, we conclude that there always exists an optimal pairing that pairs the first half with the second half. □

Using this fact, we can associate each assignment from the first half with a function f; thus, the values

$$f(1),\ f(2),\ \ldots,\ f\Big(\frac{m}{2}\Big)$$

form a bijection that assigns to the element i an element from the second half, namely, an element $\frac{m}{2} + j$ (so that $f(i) = \frac{m}{2} + j$).

Claim 6.2. *There exists a sequence*

$$a_{f(1)},\ a_{f(2)},\ \ldots,\ a_{f\left(\frac{m}{2}\right)}$$

that minimizes R and is non-increasing.

Proof. Suppose, for the sake of contradiction, that there does not exist an optimal non-increasing sequence. Then, consider an optimal assignment such that for some $i < j$ we have

$$a_{f(i)} < a_{f(j)}.$$

This implies that

$$a_i + a_{f(i)} < a_j + a_{f(j)}.$$

However, we also have

$$a_i + a_{f(j)} \le a_j + a_{f(j)} \quad (\text{since } a_i \le a_j),$$
$$a_j + a_{f(i)} < a_j + a_{f(j)} \quad (\text{by the assumed condition}).$$

Therefore, we can progressively reduce the cost by exchanging partners. Moreover, by choosing f so that it has the fewest inversions, we reach a contradiction.

Thus, the given assignment according to f is always optimal. □

For the odd case, for now, we will simply state that an extra set is used with a single element. Note that it is obvious that the largest element must go there; otherwise, it would always be interchangeable.

Now, it's easy to see that if we fix the length k of the subarrays to be considered, then the range of possible start positions is going to be either empty or $[1, p]$ for some p (we can assume $p = 0$ for the empty case), let's denote this by the function $g(k) = p$.

Note that for a given k, the subarray $[g(k),\ g(k) + k - 1]$ is valid. Thus, if we wish to relate $g(k)$ to $g(k + 2)$, we can consider the position $g(k) - 1$ as the first possible candidate, since the subarray $[g(k),\ g(k) + k + 1]$ cannot be valid (otherwise, $g(k)$ would not be maximal). This indicates that $g(k + 2) \le g(k)$, with equality occurring only when $g(k) = g(k + 2) = 0$. Furthermore, for any position $i \in [1,\ g(k) - 1]$ it suffices to verify that the sum of the endpoints of the subarray $[i,\ i + k + 1]$ does not exceed x, since the remainder of the subarray has already been verified in the k-iteration.

This suggests that we could compute the values $g(x)$ for every even/odd x in $O(N)$ using two pointers with the following algorithms:

Algorithm 1: COMPUTE-EVEN(A, N)

```
last ← N
for s ← 1 to ⌊N/2⌋ do
    k ← 2 × s
    pos ← min{max{0, last − 1}, N − k + 1}
    while pos > 0 and A_pos + A_{pos+k−1} > L do
        pos ← pos − 1
    g(k) ← pos
    last ← pos
```

Algorithm 2: COMPUTE-ODD(A, N)

```
last ← N
while last > 0 and A_last > L do
    last ← last − 1
g(1) ← last
for s ← 1 to ⌊(N−1)/2⌋ do
    k ← 2 × s + 1
    pos ← min{max{0, last − 1}, N − k + 1}
    while pos > 0 and max{A_pos + A_{pos+k−2}, A_{A_{pos+k−1}}} > L do
        pos ← pos − 1
    g(k) ← pos
    last ← pos
```

After processing the g values, we can compute the answer by:

$$result = \sum_{\substack{k=1 \\ g(k)\geq 1}}^{N} \sum_{i=1}^{g(k)} A_{i+k-1} - A_i = \sum_{\substack{k=1 \\ g(k)\geq 1}}^{N} \sum_{i=k}^{g(k)+k-1} A_i - \sum_{i=1}^{g(k)} A_i$$

If we precompute the prefix sums of A as S_i, then we can reduce the summation to:

$$result = \sum_{\substack{k=1 \\ g(k)\geq 1}}^{N} \sum_{i=k}^{g(k)+k-1} A_i - \sum_{i=1}^{g(k)} A_i = \sum_{\substack{k=1 \\ g(k)\geq 1}}^{N} S_{g(k)+k-1} - S_{k-1} - S_{g(k)}$$

Therefore, we can answer each query in $O(N)$.

Problem Tutorial: "IEEE754 Emulator"

Author(s): Oded Margalit

Keywords: **Implementation**

For this problem we must notice that the only command that is not as easy as it seems is the Fused Multiply Add one, since it requires a multiplication and an addition without step roundings between them. We could just implement the functions from scratch using unsigned integers and some heavy-implementation methods. The other possibility is to use the already implemented FMA function from C/C++ and solve the problem without anything excessively heavy to implement.

Problem Tutorial: "Triumvirates"

Author(s): CS Academy

For small values of N (even up to 51, as in the first two tests), an exhaustive search of triplets is feasible, provided that it is optimized to prune expensive branches.

For larger N, clustering techniques—such as KD-trees or Hilbert curve-like ordering, which maintain locality in ranges of consecutive points—can be employed. Once an initial solution is generated (using a greedy technique or even an exhaustive search on small clusters), an optimization such as simulated annealing (swapping random nearby triples to see if the solution improves) can be applied. It is recommended to use the entire runtime available for a test case before finalizing the solution.

Problem Tutorial: "Stick"

Author(s): Jauhar Wibisono

Keywords: **Math**

To solve this problem, we can consider that we have already added the area of the first square, and then we will add $N-1$ times the non-overlapping contribution of the remaining squares.

It is not difficult to conclude that the formula for the answer is:

$$\text{Ans} = 4L^2 + (N-1) \times \min\{2L, K\} \times (4L - \min\{2L, K\})$$

This can be computed in $O(1)$.

Problem Tutorial: "Increasing-decreasing permutations"

Author(s): Nafis Sadique

Keywords: **Dynamic programming, bitmask, preprocessing**

First, we solve an arbitrary permutation. For each number in the permutation, let's count the strictly larger numbers that came before it (call this A) and the strictly larger numbers that are after it (call this B). Now, consider the smallest number: it will either be placed at the front or at the back. Next, consider the second smallest number: it will either go to the front or the back and occupy the next available position.

In the final increasing-decreasing permutation, across all the numbers, half will have to move to the front, and the other half will have to go to the back. The numbers that go to the front will require exactly A swaps, and similarly, the numbers that go to the back will require exactly B swaps. Therefore, we just need to optimize the total number of swaps. One way to do this is to sum up the values of B for each number and then find the smallest $A - B$ for exactly half of the numbers by sorting them.

Now, since N is still quite large to generate all possible permutations, we can use a bitmask DP with the already placed numbers as the state. However, we don't have a direct way to determine which values would be picked for $A - B$. To address this, we can iterate over $A - B$ (call it C) and keep track of a counter for the number of $A - B$ values that are less than or equal to C. This approach allows us to calculate the required value efficiently.

Since this idea might not be sufficient to pass the time limit given the constraints, the expected solution requires to preprocess (this should take a few hours at most) the exact answer for every possible N and then submit a solution that computes the answer modulo M in $O(\log(answer))$.

Problem Tutorial: "Cheap Construction"

Author(s): Jauhar Wibisono

The main idea is to consider each substring of S as the string to be sent. For each substring T, we can go through its interval of occurrences in S from left to right to count k, the number of resulting connected components. We can then use the length of T as a candidate for the k-th answer.

Note: we can ignore strings that are not a substring of S since they produce the same result as 1-character strings, and we prefer the latter.

The trickiest part of the above solution is to collect interval of occurrences for each substring, preferably in left-to-right order, in reasonable time. One way to do it is by utilizing suffix array as follows.

1. Generate suffix array of S. Ideally, the suffix array contains starting positions of each suffixes and the length of the longest common prefix (LCP) of each adjacent suffixes in the suffix array.
2. Go through LCP values h from the largest ($N-1$ to 0). Maintain adjacent groups of suffixes whose LCPs are at least h. This can be done with a DSU.
3. Update answer using each combination of h and group of suffixes. The value of h gives us the subarray length. The starting positions of the suffixes in the group combined with h gives us the interval of occurrences of the substring.

A careful implementation can yield $O(N)$ processing for one value of h, resulting in a total time complexity of $O(N^2)$. An extra log factor might also be OK depending on the constant factor.

Problem Tutorial: "Disparate Data Sets"

Author(s): Douglas Gischlar

Keywords: **Implementation, parsing**

For this problem we only need a CSV parser from any language that already has it or implement one by ourselves. After implementing it correctly, we only need to filter the records as the statement says, this can be done in $O(N \log N)$ or $O(N^2)$ since the number of records N is at most 360.

The main difficulty in this problem was the CSV parsing of the records for the languages which don't have a native function for it. This can be done in linear time by splitting the string s using two pointers. Let l be the position of the first character in the current substring and $r \leftarrow l$ be a variable that is going to increase until we reach the end of the substring. We will need to increase r according to the following scenarios:

- If s_l is a double quotation mark, then we must increase r until reaching the end of the string or the next comma **such that it has an even number of double quotation marks to its left until position** l. This is correct because the only double quotation marks that will appear inside the titles will be doubled always.
- Otherwise, increase r until reaching the end of the string s or the next comma.

We can reduce both scenarios to the first one, since in the second one there won't be any double quotation marks, keeping its number to 0 (even).

After increasing r and parsing the substring, we just assign $l \leftarrow r + 1$ and go on.

Problem Tutorial: "Queries"

Author(s): CS Academy

Keywords: **SQRT Decomposition**

We will present a solution that does not support the second type of queries. Support for the second type of queries can be added by maintaining a second copy of the same structure for the sequence a_{p_i} and the permutation $P' = P^{-1}$.

Let us divide the sequence into blocks of length $B = \sqrt{N}$ and build the standard SQRT Decomposition to support range addition and range sum queries. Now, we need to add support for range permutation addition updates and ensure compatibility with range sum queries.

We will introduce an additional array to track how much was added to each block when calculating the sum of the whole block. We can use SQRT prefix sum arrays to determine how many elements in a given interval of a permutation lie inside a specific block.

Additionally, we will maintain a second SQRT Decomposition structure to handle how much was added to individual positions via the second type of updates. (Note that a segment tree cannot be used here since we require $O(1)$ value reads.)

This approach results in a solution with complexity $O((N+Q)\sqrt{N})$.

Problem Tutorial: "Icarus"

Author(s): CS Academy

Keywords: **Constructive algorithms**

To solve this problem, we note that it is always possible to construct a valid solution using a simple path. For simplicity, we assume that all nodes have only left children. This means that we will ignore all commands of type R, although we must consider some special cases before presenting our final solution.

- **Case 1:** If the given string consists solely of the letter R, then we can construct any configuration using two nodes.
- **Case 2:** If the given string contains only a single character, then we construct the tree based on the following:
  - If $S_0 = U$, then we can build the path with two nodes where $A = 1$ and $B = 2$.
  - Otherwise, we build the path with two nodes where $A = 2$ and $B = 1$.

Excluding these cases, we can compute the maximum upward distance (max_up) and the maximum downward distance (max_down) from the starting node. These values can be calculated in $O(|S|)$ time while keeping track of the final height, level.

We then construct a path with

$$N = \text{max_up} + \text{max_down} + 2$$

nodes, which does not exceed $2|S|$ in the cases under consideration. Next, we assign the nodes A and B based on the value of level:

- If level < 0, this indicates that Icarus will always move upward after executing the commands, so we set $A = \text{max_up} + 1$ and $B = N$.
- Otherwise, Icarus will always move downward or remain in the same position, so we set $A = \text{max_up} + 2$ and $B = 1$.

It is straightforward to verify that this construction is valid.

Problem Tutorial: "Doubled Sequence"

Author(s): Racsó Galván

Keywords: **Constructive algorithms**

First of all, we must consider when a solution does **not** exist:

Lemma 15.1. *Given an integer N, there is no solution for the cases*

- *$N = 4k + 2$, where k is a nonnegative integer.*
- *$N = 4k + 3$, where k is a nonnegative integer.*

Proof. If we denote the two positions in which each number appears by L_i and R_i, with $R_i = L_i + i$, it is not difficult to notice that if we sum all the positions:

$$\sum_{p=1}^{2N} p = \sum_{i=1}^{N} L_i + R_i = \sum_{i=1}^{N} L_i + (L_i + i) = \sum_{i=1}^{N} 2L_i + i$$

Taking the values modulo 2, we obtain the following equality:

$$\sum_{p=1}^{2N} p \equiv \sum_{i=1}^{N} 2L_i + i \equiv \sum_{i=1}^{N} i \pmod 2$$

It is easy to notice that the term on the left-hand side can be reduced to N modulo 2, while the right-hand side can be reduced to the number of odd numbers less than or equal to N (let us denote this quantity as $f(N)$):

$$N \equiv f(N) \pmod 2$$

If we analyze the possible values of N modulo 4, we have that:

- $N = 4k$ **satisfies** the equality. $4k \equiv 2k \equiv 0 \pmod 2$.
- $N = 4k + 1$ **satisfies** the equality. $4k + 1 \equiv 2k + 1 \equiv 1 \pmod 2$.
- $N = 4k + 2$ **does not satisfy** the equality. $4k + 2 \not\equiv 2k + 1 \pmod 2$.
- $N = 4k + 3$ **does not satisfy** the equality. $4k + 3 \not\equiv 2k + 2 \pmod 2$.

□

Now we will present a way to construct solutions for the two cases that have not been discarded.

Case $N = 4k$

It is easy to note that for $k = 1$, a possible solution is $(2, 3, 2, 4, 3, 1, 1, 4)$; now we will analyze the case $k > 1$. We denote by A^R the sequence A with its elements reversed.

First, consider the sequence of odd numbers from 1 to $N - 3$. If we denote this sequence by O, then $O^R O$ has the correct distances for each pair of values $x \in O$. On the other hand, if E is the sequence of even numbers from 2 to N, then

$$E^R \cdot x \cdot E$$

has the correct distances for each pair of values $x \in E$.

This suggests that we could analyze swapping the positions of some pair of values $x \in O$ in order to form the sequence

$$E^R \cdot x \cdot E.$$

A possible choice is to consider the values

$$c = \frac{N}{2} - 1 \quad \text{and} \quad d = N - 1$$

for this purpose. If we consider that $O^R O$ can be decomposed in the form:

$$O^R O = \alpha \cdot c \cdot \beta \cdot 1 \cdot 1 \cdot \beta^R \cdot c \cdot \alpha^R,$$

we set aside the pairs of 1 since these must always be placed consecutively, which does not give us enough flexibility to rearrange them.

It is important to note that

$$|\alpha| = \frac{N - 3 - \left(\frac{N}{2} - 1\right)}{2} = \frac{N}{4} - 1$$
$$|\beta| = \frac{\frac{N}{2} - 3 - 1}{2} = \frac{N}{4} - 2$$

Thus, the distance between α and α^R must equal $\frac{N}{2}$, while the distance between β and β^R must be 2.

Since the distance between β and β^R must be less than the distance between α and α^R, this suggests that a portion of the final solution has the form

$$lhs = X \cdot \alpha \cdot Y \cdot \beta \cdot Z \cdot \beta^R \cdot V \cdot \alpha^R \cdot W,$$

where $|Z| = 2$ and

$$|Y| + \left(\frac{N}{4} - 2\right) + |Z| + \left(\frac{N}{4} - 2\right) + |V| = \frac{N}{2},$$

which leaves us with

$$|Y| + |Z| + |V| = 4,$$

and since $|Z| = 2$, we conclude that

$$|Y| + |V| = 2.$$

On the other hand,

$$|X| + \left(\frac{N}{4} - 1\right) + |Y| + \left(\frac{N}{4} - 2\right) + 2 + \left(\frac{N}{4} - 2\right) + |V| + \left(\frac{N}{4} - 1\right) + |W| \leq 2|O| + 2 = 2\left(\frac{N}{2} - 1\right) + 2,$$

$$|X| + N - 4 + |Y| + |V| + |W| \leq N - 2 + 2,$$

$$|X| + N - 2 + |W| \leq N,$$

$$|X| + |W| \leq 2,$$

where we add 2 to the right-hand side of the inequality due to the two occurrences of d that are not counted in $O^R O$.

Now, it is easy to notice that the only candidate for the x in

$$E^R \cdot x \cdot E$$

is $N - 1$, because $|E| = \frac{N}{2}$.

This means that there must be a distance of $\frac{N}{2} - 1$ between the right end of lhs and the first occurrence of d.

On the other hand, we can note that one of the following equalities must hold:

$$X = 1 \cdot 1 \quad \text{or} \quad W = 1 \cdot 1,$$
$$Y = 1 \cdot 1 \quad \text{or} \quad V = 1 \cdot 1,$$
$$Z = 1 \cdot 1,$$

and moreover, d must appear in Z, V, or W.

With some trial and error we can conclude that

$$X = \emptyset,$$
$$Y = 1 \cdot 1,$$
$$Z = c \cdot d,$$
$$V = \emptyset,$$
$$W = c.$$

Therefore,

$$lhs = \alpha \cdot 1 \cdot 1 \cdot \beta \cdot c \cdot d \cdot \beta^R \cdot \alpha^R \cdot c.$$

Thus, the final result is

$$result = \alpha \cdot 1 \cdot 1 \cdot \beta \cdot c \cdot d \cdot \beta^R \cdot \alpha^R \cdot c \cdot E^R \cdot d \cdot E.$$

It is easy to see that this sequence is valid.

Case $N = 4k + 1$

For the case $4k + 1$, a similar analysis to that of $4k$ can be carried out, but with $c = \lfloor \frac{N}{2} \rfloor - 1$ and $d = N$. We also consider the case when $k = 1$ and build the answer $(3, 5, 2, 3, 2, 4, 5, 1, 1, 4)$ and apply the proposed solution when $k > 1$.

Furthermore, the final result will be:

$$result = \alpha \cdot 1 \cdot 1 \cdot \beta \cdot d \cdot c \cdot \beta^R \cdot \alpha^R \cdot c \cdot E^R \cdot d \cdot E.$$

The answer can be computed with an overall complexity of $O(N)$.

Problem Tutorial: "Power of three"

Author(s): CS Academy

Keywords: **Hashing, randomized algorithms**

This problem of course could be solved by calculating a logarithm on a big integer. However, this is probably too slow and tedious to code.

We present a simple-to-code randomized linear solution to this problem.

The fact that we are given a decimal representation of N allows us to read this number modulo m for arbitrary m by iterating digit-by-digit.

Lets pick 3 primes p_1, p_2, p_3 and compute:

$$r_1 \equiv N (\bmod p_1)$$
$$r_2 \equiv N (\bmod p_2)$$
$$r_3 \equiv N (\bmod p_3)$$

If there exists such x, then $3^x \mod p_i = r_i$ for every $i = 1, 2, 3$.

It can be proven that if there is no such number x with very high probability there is no such x, that the equation holds.

We know that x is bounded by $3 \cdot 10^7$ so we can check for every possible x.

Problem Tutorial: "Halving"

Author(s): CS Academy

Keywords: **Dynamic programming**

Let's first analyze the different types of pairs that can occur in positions $(2i - 1)$ and $2i$:

- If both entries A_{2i-1} and A_{2i} are given, they must satisfy the rule:

$$\begin{cases} \min(A_{2i-1}, A_{2i}) = B[i] & \text{if } R[i] = 0, \\ \max(A_{2i-1}, A_{2i}) = B[i] & \text{if } R[i] = 1. \end{cases}$$

  Otherwise, the answer is 0.

- If only one entry is given (denote this value by a), the rule imposes a constraint on the missing value depending on the following scenarios:

  - If $B[i] = a$, then there is a constraint on the other value.
  - Otherwise, the pair must consist of $B[i]$ and a, and we must verify that they satisfy the corresponding rule. If the rule does not hold, the answer is 0.
- If both entries are missing, although no number is immediately fixed, the pair imposes a constraint; this case contributes a factor of 2 due to the two possible orderings of the values (denote the product of these factors by M).

Let S be the set of numbers not used by the known entries. The remaining uncertainty is represented by:

- A **free item**: a free number from S (this item is considered to be of type 2).
- A **constraint**: a requirement arising from a pair with a missing entry. Its type depends on the rule:
  - Type 0: the constraint (from a pair with one missing entry) that is to be satisfied by a free number that appears later (or is stored for future matching).
  - Type 1: a constraint that is satisfied immediately by matching with one of the already stored free numbers.

We then combine all free items and constraints into one sorted list (ordered by numerical value) to preserve the necessary order. Define the DP state:

$$dp[j][k],$$

where:

- j is the number of free items that are stored (available for pairing), and
- k is the number of pending constraints (from type 0) waiting for a free number.

As we process each item in the sorted list, we update the DP state according to the item type:

- **For a free item (Type 2):**
  1. *Store it:* Increase the free count:
  $$new_dp[j+1][k] \mathrel{+}= dp[j][k].$$
  2. *Match it with a pending constraint:* If $k > 0$, assign it to one of the k pending constraints:
  $$new_dp[j][k-1] \mathrel{+}= k \times dp[j][k].$$
- **For a constraint of Type 0:** This constraint is stored as pending, so:
$$new_dp[j][k+1] \mathrel{+}= dp[j][k].$$
- **For a constraint of Type 1:** Here, a free number that has already been stored must be used to satisfy the constraint. If $j > 0$, then:
$$new_dp[j-1][k] \mathrel{+}= j \times dp[j][k].$$

The transitions are applied sequentially as we process each item and we replace dp with new_dp in each iteration. At the end, $dp[0][0]$ gives the number of valid configurations.

The total number of valid permutations is

$$\text{Answer} = M \times dp[0][0] \pmod{998244353},$$

where M is the multiplicative factor from pairs with both entries missing.

The time complexity is $O(N^3)$.

Problem Tutorial: "King's Order"

Author(s): Philip Garabandic

Keywords: **Topological sort**

For this problem we can just use a variation of the topological sort algorithm with BFS but using a priority queue to follow the kingdom's ordering when we have multiple possible projects to use. We will use a as the group id array.

Algorithm 3: KINGS-ORDER(V, E, a)

```
Let in[1, ..., |V|] be an array filled with 0
for (u, v) ∈ E do
    in[v] ← in[v] + 1
Let Q be a min priority queue of tuples
for u ∈ V do
    if in[u] = 0 then
        ENQUEUE(Q, (a[u], u))
res ← {}
while Q ≠ ∅ do
    (g, u) = EXTRACT-MIN(Q)
    res ← res ∪ {u}
    for (u, v) ∈ E do
        in[v] ← in[v] − 1
        if in[v] = 0 then
            ENQUEUE(Q, (a[v], v))
if |res| ≠ |V| then
    return ∅
else
    return res
```

It is easy to see that this algorithm has $O(M \log N)$ complexity.

It's possible to just use an array and a linear scan to get the minimum enqueued value since N is small, this alternate idea would have an $O(N^2 + M)$ complexity.

Problem Tutorial: "Balls"

Author(s): CS Academy

Keywords: **Inclusion-exclusion principle, dynamic programming, math**

It is easy to come up with an inclusion-exclusion solution that runs in $O(2^k)$. This solution operates only on numbers of the form $\left\lfloor \frac{N}{c} \right\rfloor$. A well-known lemma states that there are only $O(\sqrt{N})$ different values with the form $\left\lfloor \frac{N}{c} \right\rfloor$.

We could do a knapsack-like dynamic programming that builds every possible $(-1)^{k+1} \left\lfloor \frac{N}{c} \right\rfloor$, where c is the product of k values E_i. It is easy to notice that this solution has $O(k\sqrt{N})$ different states, so we will maintain only the visited states $(\left\lfloor \frac{N}{c} \right\rfloor, sign)$ by layer and sort E by decreasing value. This reduces the number of existing states in the initial phases of the algorithm and allows us to merge the generated values with the previous ones in linear time (since both sequences are sorted).

The algorithm is the following:

Algorithm 4: BALLS-SOLUTION(E, N, k)

Sort E by decreasing value
$states \leftarrow \emptyset$
for $i \leftarrow 1$ **to** k **do**
  $new_states \leftarrow \emptyset$
  for $(value, sign) \in states$ **do**
    if $value \geq E_i$ **then**
      $new_states \leftarrow new_states \cup \left\{\left(\left\lfloor \frac{value}{E_i} \right\rfloor, -sign\right)\right\}$
  $new_states \leftarrow new_states \cup \left\{\left(\left\lfloor \frac{N}{E_i} \right\rfloor, 1\right)\right\}$
  $states \leftarrow$ MERGE$(states, new_states)$
$ans \leftarrow 0$
for $(value, sign) \in states$ **do**
  $ans \leftarrow ans + value \times sign$
return ans

Where the MERGE is a function that returns the result of linearly merging two sorted arrays.

Problem Tutorial: "Corporation"

Author(s): CS Academy

Keywords: **Segment tree, lazy propagation**

We will use a segment tree. Let us maintain the maximum, minimum, and sum for each interval. It is straightforward to process the second type of update while maintaining such a segment tree. When we encounter a first type of query, we descend into all nodes where max = min. This condition indicates that the entire interval consists of identical elements, allowing us to determine how the happiness of employees in this interval changes.

Additionally, we will maintain an auxiliary add/sum segment tree to track the happiness of the employees.

We can prove that this descent does not affect the overall complexity by considering a potential function equal to the number of maximal intervals of identical elements. The final complexity is $O((N + Q) \log^2 N)$. This can be reduced to a single $\log N$ by merging updates to the second segment tree, but the improvement is minor and, in practice, only speeds up the process by a factor of two.

Problem Tutorial: "This is not an optimization problem"

Author(s): Racsó Galván

Keywords: **Fast fourier transform, combinatorics**

For this problem, we must first notice that once we define a subset of terminals, the answer of its Steiner Tree consists of all the nodes x such that the removal of x disconnects at least one node from the set. We will call all the sets that do not become disconnected after the removal of x as *unaffected sets*. This gives us the idea of analyzing the contribution for each node as:

$$\text{contribution}(x, k) = W_x \times (\text{total sets} - \text{unaffected sets})$$

The first value is $\binom{n}{k}$ and the second one can be computed as the number of ways of choosing k nodes from the same connected component after removing x. That means that if the sizes of the connected components after removing x are $sz_{x,1}, \ldots, sz_{x,m_x}$, then the contribution of x is:

$$\text{contribution}(x, k) = W_x \times \left(\binom{N}{k} - \sum_{i=1}^{m_x} \binom{sz_{x,i}}{k}\right)$$

We can distribute the factor:

$$\text{contribution}(x,k) = W_x \times \binom{N}{k} - W_x \times \sum_{i=1}^{m_x} \binom{sz_{x,i}}{k}$$

Notice that we can sum up this contribution for all nodes with a fixed k:

$$\sum_{x=1}^{N} \text{contribution}(x,k) = \sum_{x=1}^{N} W_x \times \binom{N}{k} - W_x \times \sum_{i=1}^{m_x} \binom{sz_{x,i}}{k}$$

And separate the known values:

$$\sum_{x=1}^{N} \text{contribution}(x,k) = S \times \binom{N}{k} - \sum_{x=1}^{N} W_x \times \sum_{i=1}^{m_x} \binom{sz_{x,i}}{k}$$

Where S is the sum of weights of all the nodes.

Now, we can group all the binomial coefficients that have the same size and ignore the ones which are less than k (since $\binom{s}{k} = 0$ for such s), transforming our expression to:

$$\sum_{x=1}^{N} \text{contribution}(x,k) = S \times \binom{N}{k} - \sum_{s=k}^{N} C_s \times \binom{s}{k}$$

Where C_s will be the sum of $W_x \times frec(x,s)$, where $frec(x,s)$ is the number of connected components of size s formed after removing x.

This expression can be easily computed in $O(N^2)$. However, this doesn't suffice to get the full points.

Notice that we can replace the binomial coefficient with the factorials involved:

$$\sum_{x=1}^{N} \text{contribution}(x,k) = S \times \binom{N}{k} - \sum_{s=k}^{N} C_s \times \frac{s!}{k!(s-k)!}$$

We can take out the $k!$ from the denominator:

$$\sum_{x=1}^{N} \text{contribution}(x,k) = S \times \binom{N}{k} - \frac{1}{k!} \sum_{s=k}^{N} C_s \times \frac{s!}{(s-k)!}$$

We can see that s and $(s-k)$ have a difference of exactly k and we can relate a difference with a sum (just multiplying the second term by -1), so we will define the following polynomials:

$$P(x) = \sum_{s=0}^{N} (C_{N-s} \times (N-s)!)\, x^s$$

$$Q(x) = \sum_{s=0}^{N} \frac{1}{s!} x^s$$

And if we compute their product $R(x) = P(x) \times Q(x)$, then its $(N-k)$-th term will be:

$$R(x)_{N-k} = \sum_{j=0}^{N-k} P(x)_j \times Q(x)_{N-k-j}$$

$$R(x)_{N-k} = \sum_{j=0}^{N-k} (C_{N-j} \times (N-j)!) \times \frac{1}{(N-k-j)!} = \sum_{j=k}^{N} C_j \times \frac{j!}{(j-k)!}$$

Which is what we wanted. We can use an FFT to compute the polynomial multiplication in $O(N \log N)$ and then iterate over each k to compute their answers.

Problem Tutorial: "Digits swap"

Author(s): CS Academy

Keywords: **Bitmask, brute force**

Important note: This problem seems like a simple greedy but in fact is NP.

Consider some optimal solution. It contains a set of swaps on pairs of positions. Let's consider an undirected graph, where nodes are the positions and the edges are the information that digits at these two positions were swapped at some point. If two positions are in different components of this graph, then digits from positions from one component couldn't travel to the other one regardless of the order of swaps.

One component with X vertices can be "sorted" using at most $X-1$ swaps bringing it to it's optimal state (Notice that we can model it as a permutation and the number of optimal swaps is actually $X-c$ where c is the number of cycles, but considering $X-1$ swaps doesn't break the structure since we could achieve the $X-c$ swaps by partitioning this component into its cycles and adding them up). This amount of swaps is also necessary for it to be a component, so any optimal solution can be transformed into one where we use $X-1$ swaps per component of size X and each component is brought to its optimal state using no additional operations and obtaining not a worse result. Thus we will consider only solutions of this kind.

We can define $dp(mask, cnt)$ as the maximum difference after sorting the digits in $mask$ using exactly cnt swaps. In addition, we will define $dif(mask)$ as the difference after sorting the digits in $mask$ with respect to the original number. Notice that:

$$dp(mask, cnt) = \max_{\substack{m \subseteq mask \\ |m|-1 \leq cnt}} \{dp(mask \oplus m, cnt - |m| + 1) + dif(m)\}$$

Where $\oplus$ is the bitwise XOR operation.

This can be computed in $O(3^N \cdot N)$ with a preprocessing of $O(2^N \cdot N)$ for precomputing the difference for each possible $mask$.

Problem Tutorial: "Brick stacks"

Author(s): CS Academy

Keywords: **Greedy, sortings**

For this problem we can sort the bricks in non-decreasing order and find the longest interval $[l, r]$ where $a_r < a_l + x$. The length len of this interval is a lower bound for the answer and it's satisfiable. We can just assign bricks $a_i, a_{i+len}, a_{i+2len}, \ldots$, to one stack and the conditions are satisfied. This solution runs in $O(N \log N)$.

Problem Tutorial: "Stones"

Author(s): CS Academy

Keywords: **Dynamic programming, probabilities**

We will use dynamic programming. Let us define $dp(a, b, c, d)$ as the probability of the first player winning when he has a blue and b red stones, and the second player has c blue and d red stones.

The only strategy the first player can take here is "Pick red with probability p. Otherwise, pick blue" for some p. The same applies to the second player with probability q.

This creates a metagame where both players choose a number from $[0, 1]$ and the payout is $f(p, q) = Xpq + Yp(1-q) + Zq(1-p) + V(1-p)(1-q)$, for some values X, Y, Z, V which depend only on the states with smaller $a + b + c + d$ (so we can iterate through the states in increasing order of $a + b + c + d$). Player 1 wants to maxmin and player 2 wants to minmax the result. We know that minmax = maxmin here, so we can just calculate maxmin (it is a nash equilibrium in this game), and that for a fixed p, the "worst" q is either

0 or 1, so we want to pick a p that maximizes $\min(f(p,0), f(p,1))$, which is the minimum of two linear functions and can be calculated by considering a few cases.

It is easy to see that the probability that the current player wins is $1 - \mathbb{P}(\text{next player wins})$, so:

$$\begin{aligned} X &= 1 - dp(c, d, a-1, b) \\ Y &= 1 - dp(c-1, d, a, b) \\ Z &= 1 - dp(c, d-1, a, b) \\ V &= 1 - dp(c, d, a, b-1) \end{aligned}$$

Then, we have

$$\begin{aligned} f(p,0) &= Yp + V(1-p) \\ f(p,1) &= Xp + Z(1-p) \end{aligned}$$

Since we want to maximize the mininum of both lines, then we must use the p in their intersection:

$$p^* = \frac{V - Z}{X - Z - Y + V}$$

Finally, our answer will be $f(p^*, 0)$ for the current state.

Problem Tutorial: "Rectangles and arrays"

Author(s): Iván Bórquez

Keywords: **Divide and conquer**

For this problem, we can design a simple Divide and Conquer algorithm to compute the maximum rectangle possible. Let's define $f(l, r)$ as the maximum rectangle that can be computed from positions $[l, r]$. Then, we have some cases:

- If $l = r$, then the answer is $\max\{a_l, X\}$
- Otherwise, we can split the range in half using the position $mi = \lfloor \frac{l+r}{2} \rfloor$. Then, we will compute $f(l, mi)$ and $f(mi + 1, r)$.

  The only subarrays that are pending to be processed are the ones that start in $[l, mi]$ and end in $[mi + 1, r]$, so they are a suffix of $[l, mi]$ and a prefix of $[mi + 1, r]$.

  Like in merge sort, we will need to merge the information of all suffixes of the left part and all the prefixes of the right part.

  Let b_i be the minimum value associated with position i during this process — if $i \leq mi$ then b_i is minimum of the suffix $[i, mi]$, otherwise it is the minimum of the prefix $[mi + 1, i]$, **without** changing any position by X. We will define c_i as well but it computes the minimum value changing at most one of its positions by X.

  We should notice that both b_i and c_i are monotone in the ranges $[l, mi]$ and $[mi + 1, r]$, so we can just use two pointers to compute the crossed values between the valid left and right parts (b with b or b with c).

Since the merging part is $O(N)$, we will have the same complexity as in Merge sort, which is $O(N \log N)$.

Note: There is an $O(N)$ solution using stacks, but the $O(N \log N)$ solution suffices to pass the time limit.

Problem Tutorial: "Invertible Pairs"

Author(s): Sebastian Sánchez

Keywords: **Dynamic programming**

For this problem we can define a function $dp(i, t)$ that is going to compute the maximum subarray sum ending in position i given that if:

- $t = 1$: We applied once the multiplication by -1 in positions $2 \times \lceil \frac{i}{2} \rceil - 1$ and $2 \times \lceil \frac{i}{2} \rceil$.
- $t = 0$: We haven't applied the multiplication by -1 in positions $2 \times \lceil \frac{i}{2} \rceil - 1$ and $2 \times \lceil \frac{i}{2} \rceil$.

So, we will compute the answer depending on the parity of i:

- If i is even:
  - If $t = 0$ we can only have the following subarrays types:
    - Length 0: 0
    - Length 1: a_i
    - Length 2: $a_i + a_{i-1}$
    - Length 3 or more: $a_i + a_{i-1} + \max\{dp(i-2, 0), dp(i-2, 1)\}$.
  - If $t = 1$ we can only have the following subarrays types:
    - Length 0: 0
    - Length 1: $-a_i$
    - Length 2: $-a_i - a_{i-1}$
    - Length 3 or more: $-a_i - a_{i-1} + \max\{dp(i-2, 0), dp(i-2, 1)\}$.
- If i is odd:
  - If $t = 0$ we can only have the following subarrays types:
    - Length 0: 0
    - Length 1: a_i
    - Length 2 or more: $a_i + \max\{dp(i-1, 0), dp(i-1, 1)\}$.
  - If $t = 1$ we can only have the following subarrays types:
    - Length 0: 0
    - Length 1: $-a_i$
    - Length 2 or more: $-a_i + \max\{dp(i-1, 0), dp(i-1, 1)\}$.

If we compute the function with memoization, we will have a final complexity of $O(N)$ per test case. Assume that $f(0, 0) = f(0, 1) = 0$.

Problem Tutorial: "Sierpinski"

Author(s): CS Academy

Keywords: **Recursion**

For this problem we can just focus on making a recursion $f(x, y, d, L)$ that computes the color of the cell in position (x, y) inside of a triangle of depth d and height L.

We can notice that if $d = 1$, then the answer is always 1 (since all its points are red). Otherwise, we will recurse. Let $l = \frac{L+1}{2}$, then:

- If $x < l$, then we can just compute $f(x, y, d-1, l-1)$.
- If $x = l$, then the point is blue.
- If $y \leq x - l$, then we can compute $f(x - l, y, d-1, l-1)$.
- If $y \geq l + 1$, then we can compute $f(x - l, y - l, depth - 1, l - 1)$.
- Otherwise, the point is blue.

Notice that we can compute the answer with $f(x, y, 30, 1610612735)$ in at most 30 steps, so the complexity is $O(Q \cdot D)$ with $D = 30$.